

Assignment 3 DESIGN.pdf:

Description of Program:

This program implements Insertion Sort, Shell Sort, Heap sort, and recursive Quick sort based on the provided Python pseudocode. The interface for these sorts were given as the header files insert.h, shell.h, heap.h, and quick.h. The test harness creates an array of pseudorandom elements and tests each of the sorts. Program will gather statistics about each sort and its performance. The statistics will be the size of the array, the number of moves required, and the number of comparisons required.

Files to be included in directory "asgn2" + pseudocode/structure:

- insert.c
 - Implements insertion sort
 - Insertion Sort compares the current element with each of the preceding elements in descending order until its position is found.
 - Use for loop to go through the array
 - Compare the current element (key) to its predecessor.
 - If the key element is smaller than its predecessor, compare it to the elements before. Move the greater elements one position up to make space for the swapped element.
- insert.h
 - specifies the interface to insert.c
- heap.c
 - Implements heap sort
 - Build a max heap from the input data.

- At this point, the largest item is stored at the root of the heap. Replace it with the last item of the heap followed by reducing the size of heap by 1. Finally, heapify the root of the tree (the heap needs to be fixed so that it once again obeys the constraints of a heap).
 - Repeat step 2 while the size of the heap is greater than 1.
- heap.h
 - specifies the interface to heap.c
- quick.c
 - implements recursive Quicksort.
 - Quick sort is a divide-and-conquer algorithm. It partitions arrays into two sub-arrays by selecting an element from the array and designating it as a pivot. Elements in the array that are less than the pivot go to the left sub-array, and elements in the array that are greater than or equal to the pivot go to the right sub-array.
 - Once pivot is chosen, put it at its correct position in sorted array
 - Put all smaller elements (smaller than pivot) before pivot
 - Put all greater elements (greater than pivot) after pivot
 - All this should be done in linear time.
- quick.h
 - specifies the interface to quick.c
- set.h
 - implements and specifies the interface for the set ADT.
- stats.c
 - implements the statistics module.
- stats.h
 - specifies the interface to the statistics module.

- shell.c
 - implements Shell Sort.
 - Shell Sort is a variation of Insertion Sort and first sorts pairs of elements which are far apart from each other. The distance between these pairs of elements is called a gap. Each iteration of Shell Sort decreases the gap until a gap of 1 is used.
 - Initialize the gap size
 - Divide the array into subarrays of equal gap size.
 - Apply insertion sort on the subarrays.
 - Repeat the above steps until the gap size becomes less than 1, resulting into a sorted array.
- shell.h
 - specifies the interface to shell.c
- sorting.c
 - Initialize array with pseudo random elements from random seed
 - function for printing array
 - main() takes any combination of the following command-line options and produces the sorts along with number of elements, moves, and compares.
 - -a: Employs all sorting algorithms
 - -e: Enables Heap Sort
 - -i: Enables Insertion Sort
 - -s: Enables Shell Sort
 - -q: Enables Quick sort
 - -r seed: Set the random seed to seed. The default seed should be 13371453

- -n size: Set the array size to size. The default size should be 100
- -p elements: Print out the number of elements from the array. The default number of elements to print out should be 100. If the size of the array is less than the specified number of elements to print, print out the entire array and nothing more
- -h: Prints out program usage.

Free array

- WRITEUP.pdf
 - This document must be a proper PDF.
 - Graphs displaying the difference between the values reported by implemented functions and that of the math library's.
 - Use a UNIX tool to produce graphs. gnuplot is recommended.
- Makefile
 - CC = clang must be specified.
 - CFLAGS = -Wall -Wextra -Werror -Wpedantic must be specified.
 - make must build the Mathlib-test executable, as should make all and make mathlib-test
 - make clean must remove all files that are compiler generated.
 - make format should format all your source code, including the header files.
- README.md
 - In proper Markdown syntax.
 - Describe how to use your program and Makefile
 - Lists and explains any command-line options that program accepts.
 - Reports any false positives by scan-build
 - Notes down any known bugs or errors in this file as well for the graders.
- DESIGN.pdf
 - This document must be a proper PDF.

- Describes design and design process for program with enough detail such that a sufficiently knowledgeable programmer would be able to replicate implementation.